MAJOR PROJECT REPORT ON

AlertVision - Realtime Aggression Detection using Deep Learning and Video Analytics

in the partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Submitted by:

B. Karthik Sai	**22B81A6621**
A. Siddhartha	**22B81A6642**
V. Siddhartha	**22B81A6643**

Under the guidance of
Ms. K. Sinduja
Assistant Professor
CSE(AI&ML)

CVR COLLEGE OF ENGINEERING

(***An Autonomous Institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)

Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

APRIL 2026

CVR COLLEGE OF ENGINEERING

(***An Autonomous Institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad***)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

DEPARTMENT OF

CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

CERTIFICATE

This is to certify that the Major Project Stage entitled **"AlertVision - Realtime Aggression Detection using Deep Learning and Video Analytics"** is a Bonafide record of work carried out by **B. Karthik Sai (22B81A6621), A. Siddhartha (22B81A6642)** and **V. Siddhartha (22B81A6643)** under the guidance of **Ms. K. Sinduja.** This report is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)** to **CVR College of Engineering**, affiliated to Jawaharlal Nehru Technological University, Hyderabad, during the academic year 2025-26.

Project Guide
Ms. K. Sinduja
Assistant Professor
Department of CSE (AI & ML)

Project Coordinator
Dr. A. Jagadeeswara Rao
Professor
Department of CSE (AI & ML)

Dr. R. Usha Rani
Professor & Head
Department of CSE (AI & ML)

External Examiner

CVR COLLEGE OF ENGINEERING

(*An Autonomous institution, NAAC Accredited and Affiliated to JNTUH, Hyderabad*)

Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

DECLARATION

We hereby declare that the Major Project entitled **"AlertVision - Realtime Aggression Detection using Deep Learning and Video Analytics"** is an original work carried out and submitted by us to the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), **CVR College of Engineering**, affiliated to Jawaharlal Nehru Technological University, Hyderabad, in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning). This report is a bona fide record of the project work completed under the guidance of **Ms. K. Sinduja**, Assistant Professor, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning).

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Signature of the Student
B. Karthik Sai

Signature of the Student
A. Siddhartha

Signature of the Student
V. Siddhartha

Date:

Place:

ACKNOWLEDGEMENT

We are happy to present our Major Project titled **"AlertVision - Realtime Aggression Detection using Deep Learning and Video Analytics",** completed as part of our curriculum in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) at **CVR College of Engineering**.

We respect and thank our internal guide, **Ms. K. Sinduja,** Assistant Professor, Department of CSE (Artificial Intelligence and Machine Learning), for giving us all the support and guidance, which helped us complete the project duly.

Our sincere thanks to **Dr. A. Jagadeeswara Rao, Dr. M. Surya Bhupal Rao** members of the Project Review Committee, for their valuable guidance and support, which greatly contributed to the successful completion of our project.

We would like to express heartfelt thanks to **Dr. R. Usha Rani,** Professor and Head of the Department, for providing us with an opportunity to do this project and extending support and guidance. Our gratitude also extends to **Dr. Lakshmi H.N**, Associate Dean, Emerging Technologies, for her encouragement to complete the project work.

We are thankful to our Vice-Principal**, Prof. L. C. Siva Reddy,** for providing excellent computing facilities and a disciplined atmosphere for our work.

We wish a deep sense of gratitude and heartfelt thanks to **Dr. K. Rama Mohan Reddy,** Principal, and the **Management** for providing excellent lab facilities and tools.

Finally, we are thankful for and fortunate enough to get constant encouragement, support, and guidance from all **Teaching staff, Department CSE (Artificial Intelligence and Machine Learning),** which helped us complete this project work.

TABLE OF CONTENTS

Chapter No.		Contents	Page No.
		List of Tables	i
		List of Figures	ii
		Abbreviations	iii
		Abstract	iv
1		**Introduction**	
	1.1	Motivation	1
	1.2	Problem Statement	2
	1.3	Project Objectives	3
	1.4	Project Report Organization	3-4
2		**Literature Survey**	
	2.1	Existing Works	5
	2.2	Summary of Existing Works	6-7
	2.3	Limitations of Existing Work	8
3		**Software Requirements Specifications**	
	3.1	Software Requirements	9
	3.2	Hardware Requirements	11
	3.3	Functional Requirements	11
	3.4	Non-Functional Requirements	13
4		**Proposed System Design**	
	4.1	Proposed Method	16-17
	4.2	Class Diagram	18
	4.3	Use case Diagram	19
	4.4	Activity Diagram	20-21
	4.5	Sequence Diagram	21-22
	4.6	System Architecture	23-24
	4.7	Technology Description	25
5		**Implementation & Testing**	
	5.1	Implementation	26-31
	5.2	Testing & Results	32-34
6		**Conclusion & Future Scope**	35-37
	6.1	**Paper publication Details**	
	6.2	**Project Outcome Mapping Table**	37
7		**References**	38-39
8		**Appendix**	40

LIST OF TABLES

Figure No.	Title	Page No.
2.1	REFERENCE PAPERS	06-07
3.1	SOFTWARE REQUIREMENTS	14
3.2.1	HARDWARE REQUIREMENTS	15
3.2.2	TRAINING HARDWARE REQUIREMENTS	15
5.2	TESTING OUTCOME	33

LIST OF FIGURES

Figure No.	Title	Page No.
4.2	CLASS DIAGRAM	18
4.3	USE CASE DIAGRAM	19
4.4	ACTIVITY DIAGRAM	20
4.5	SEQUENCE DIAGRAM	21
4.6	SYSTEM ARCHITECTURE	23
5.1.1	DASHBOARD	29
5.1.2	INCIDENTS PAGE	29
5.1.3	ANALYTICS	30
5.1.4	NOTIFICATION SETTINGS PAGE	30
5.1.5	CAMERA SETTINGS	31
5.1.6	ALERT MAIL	31

ABBREVATIONS

AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional Neural Network
FDA	Food and Drug Administration
GUI	Graphical User Interface
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
JWS	JSON Web Signature
JWT	JSON Web Token
LSTM	Long Short-Term Memory
ML	Machine Learning
NLP	Natural Language Processing
NLU	Natural Language Understanding
REST	Representational State Transfer
RNN	Recurrent Neural Network
RPM	Remote Patient Monitoring
TTS	Text-to-Speech
UI	User Interface
VS	Visual Studio

ABSTRACT

The escalating incidence of public altercations, combined with the rapid expansion of surveillance infrastructure, has created an urgent need for automated, real-time aggression detection in video streams. Traditional manual monitoring methods are limited by issues such as poor scalability, operator fatigue, and subjective bias, while many existing automated solutions face challenges related to computational inefficiency, environmental variability, and lack of temporal awareness. Additionally, these systems often struggle to generalize across different scenarios, lighting conditions, and crowd densities, further reducing their reliability in real-world deployments.

To address these limitations, this work introduces AlertVision, a lightweight, end-to-end framework capable of detecting physical aggression, threatening gestures, and rapid hostile movements in live surveillance feeds. The system integrates YOLOv5 for human detection, Dlib for pose landmark extraction, and a hybrid CNN-LSTM architecture for spatiotemporal classification. As a result, AlertVision achieves sub-second latency even on consumer-grade hardware, while maintaining robustness across diverse environmental conditions and complex activity patterns.

Implemented in Python using OpenCV, TensorFlow, and NumPy, the framework ensures both high accuracy and practical deployability. Furthermore, its modular design allows for future enhancements, including audio-visual fusion, real-time alert systems, and multi-camera tracking, positioning AlertVision as a scalable, efficient, and proactive solution for improving public safety and surveillance intelligence.

CHAPTER 1

INTRODUCTION

The rapid expansion of surveillance systems in modern urban environments has made continuous manual monitoring increasingly impractical. Human operators often face challenges such as fatigue, limited attention span, and subjective bias, leading to missed or delayed detection of critical events. This highlights the need for intelligent, automated solutions capable of analyzing video streams in real time and identifying potential threats efficiently.

Recent advancements in computer vision and deep learning have enabled significant progress in human activity recognition. However, detecting aggressive or violent behavior remains challenging due to environmental variations, complex motion patterns, and the need for temporal understanding. Many existing systems are either computationally expensive or lack real-time performance.

To address these limitations, this work introduces AlertVision, a lightweight framework for real-time aggression detection in surveillance video streams. By combining object detection, pose estimation, and spatiotemporal modeling, the system aims to deliver accurate, low-latency performance on consumer-grade hardware, contributing to more proactive and reliable public safety solutions.

1.1 MOTIVATION

Urbanization and the widespread presence of public surveillance systems have significantly increased the demand for intelligent, autonomous safety solutions. According to the National Crime Records Bureau (NCRB) 2023, public altercations in urban India have risen by 18% year-over-year, with 63% of incidents occurring in high-density areas such as transit hubs, shopping centers, and educational institutions. While human-operated monitoring provides contextual awareness, it is inherently constrained by cognitive fatigue, attentional lapses, and the inability to effectively process multiple video streams simultaneously. These limitations often lead to delayed responses or complete oversight of critical incidents, as highlighted by the 2022 Mumbai Metro brawl, where security personnel failed to intervene promptly due to monitoring overload. Furthermore, the rapid expansion of CCTV networks—projected to exceed 100 million cameras in India by 2025 (IHS Markit, 2024)—intensifies the need for real-time, data-driven behavioral analysis. AlertVision addresses this gap by augmenting human vigilance with automated, objective threat recognition, enabling more proactive and efficient security intervention.

1.2 PROBLEM STATEMENT

Despite advancements in computer vision, existing aggression detection systems face several critical challenges:

1. **Computational Inefficiency:** Many state-of-the-art models require high-end GPUs, making them impractical for deployment in resource-constrained or edge environments.
2. **Environmental Fragility:** System performance often degrades under varying lighting conditions, occlusions, and high crowd density, resulting in increased false positives and false negatives.
3. **Temporal Insensitivity:** Most systems analyse aggression as a static pose rather than a dynamic sequence, failing to capture the progression and evolution of hostile behavior.

AlertVision Approach to Overcome Limitations

AlertVision addresses these challenges through the following strategies:

- **Lightweight Architecture:** Optimized for real-time performance on standard, consumer-grade hardware.
- **Adaptive Preprocessing:** Designed to handle environmental variability, ensuring consistent performance across diverse conditions.

1.3 PROJECT OBJECTIVES

The core objectives of AlertVision are:

1. **Sub-Second Latency:** To achieve sub-second latency on consumer-grade hardware (e.g., Intel i5, NVIDIA GTX 1650).
2. **Robust Model Integration:** To integrate YOLOv5 for human detection, Dlib for pose estimation, and a ResNet-LSTM architecture for classification, ensuring end-to-end robustness.
3. **Adaptive Preprocessing:** To implement adaptive preprocessing techniques (e.g., histogram equalization, motion compensation) to handle lighting variations and occlusions effectively.
4. **Scalable System Design:** To structure the system for future enhancements, including audio-visual fusion and multi-camera tracking capabilities.

1.4 PROJECT REPORT ORGANIZATION

This document is organized into six chapters, each systematically addressing the development and evaluation of the AlertVision framework:

Chapter 1: Introduction

This chapter establishes the context and motivation for the study by discussing the rapid growth of surveillance systems and the limitations of manual monitoring. It clearly defines the problem of real-time aggression detection, outlines the key challenges faced by existing systems, and presents the objectives and contributions of AlertVision as a proposed solution.

Chapter 2: Literature Review

This chapter provides a comprehensive review of recent research in the fields of computer vision, human activity recognition, and aggression detection. It critically analyzes existing methodologies, highlighting their strengths and limitations, and identifies research gaps that justify the need for a more efficient and robust system like AlertVision.

Chapter 3: System Design and Architecture

This chapter details the overall architecture of the proposed system, including module-level design, data flow, and system interactions. It incorporates UML diagrams such as class diagrams, sequence diagrams, and component diagrams to illustrate the structure and workflow. The design rationale behind each component is also explained to demonstrate how the system achieves scalability and efficiency.

Chapter 4: Methodology and Implementation

This chapter focuses on the technical implementation of AlertVision. It explains the selection and integration of key models such as YOLOv5 for human detection, Dlib for pose estimation, and a CNN-LSTM architecture for spatiotemporal analysis. Additionally, it discusses data preprocessing techniques, training strategies, hyperparameter tuning, and optimization methods used to ensure real-time performance.

Chapter 5: Results and Evaluation

This chapter presents a detailed evaluation of the system's performance using both benchmark datasets (e.g., RWF-2000, Hockey Fights) and real-world scenarios. It includes quantitative metrics such as accuracy, precision, recall, and latency, along with qualitative analysis of

detection results. Comparisons with existing approaches are also provided to demonstrate the effectiveness of AlertVision.

Chapter 6: Conclusion and Future Work

This chapter summarizes the key findings and contributions of the project, emphasizing its practical significance in enhancing public safety. It also discusses current limitations of the system and proposes potential future improvements, including advanced features like audio-visual fusion, multi-camera tracking, and further model optimization for large-scale deployment.

CHAPTER 2
LITERATURE SURVEY

2.1 EXISTING WORKS

Recent advancements in aggression and violence detection have leveraged deep learning techniques to enhance accuracy and applicability in real-world surveillance environments. Senadeera et al. (2024) introduced CUE-Net, which uses spatial cropping and attention mechanisms to focus on key regions, resulting in a 12% improvement in accuracy on benchmark datasets. However, its high computational requirements limit real-time deployment on resource-constrained systems. Similarly, Comminiello et al. (2024) proposed JOSENet, integrating appearance and motion cues, achieving 94% precision on surveillance data. However, its dependence on high-end GPUs and reduced performance in low-light conditions pose practical challenges. Basak et al. (2024) developed DIFEM to model inter-limb dynamics for detecting subtle pre-aggressive behaviors, but its performance declines in crowded and highly occluded environments.

Broader analyses highlight key trends and limitations in the field. Negre et al. (2024) identified CNN-LSTM architectures as dominant for capturing temporal dynamics but noted the lack of edge-optimized solutions and limited real-world validation. Vosta and Yow (2022) demonstrated CNN-RNN effectiveness for incorporating temporal context but provided limited insight into robustness under varying environmental conditions. Vijeikis et al. (2022) proposed a lightweight CNN-LSTM model capable of processing video streams at 25 fps with minimal loss in accuracy; however, it struggles in complex and crowded scenes. Patel (2021) developed a real-time CNN-LSTM system with adaptive thresholding, reducing false positives in dense environments, but it faces scalability issues in multi-camera setups and remains sensitive to occlusions. Overall, these studies highlight significant progress while underscoring challenges in efficiency, robustness, and scalability.

2.2 SUMMARY OF EXISTING WORKS

Table 2.1: Reference Papers

Paper Name	Authors	Algorithm Used	Contribution	Limitations
CUE-Net (2024)	Senadeera et al.	Spatial cropping, attention mechanisms	Achieves 12% accuracy improvement by focusing on regions of interest in dynamic scenes	Limited real-world validation; high computational cost for attention mechanisms
JOSENet (2024)	Comminiello et al.	Joint-stream embedding (appearance + motion)	94% precision on surveillance footage via multimodal feature fusion	Requires high-end GPUs; performance drops in low-light conditions
DIFEM (2024)	Basak et al.	Keypoint-interaction module	Outperforms baselines in detecting subtle pre-aggressive behaviors via inter-limb dynamics	Limited to specific pose-based scenarios; less effective in crowded scenes
Deep Learning Review (2024)	Negre et al.	CNN-LSTM hybrids	Identifies CNN-LSTM as dominant for temporal modeling; emphasizes need for edge-optimized designs	Not an implementation; lacks specific performance metrics
CNN-RNN System (2022)	Vosta & Yow	CNN-RNN architecture	Validates temporal context for real-world surveillance, achieving robust detection	Limited discussion on environmental robustness; moderate computational demand

Lightweight CNN-LSTM (2022)	Vijeikis et al.	Optimized CNN-LSTM	Processes 25 FPS on mid-tier GPUs with minimal accuracy loss	Reduced accuracy in complex scenes; limited to single-camera setups
Real-Time CNN-LSTM (2021)	Patel	CNN-LSTM with adaptive thresholding	25 FPS on mid-tier GPUs; reduces false positives in crowded scenes	Limited scalability for multi-camera systems; sensitive to occlusions

2.3 LIMITATIONS OF EXISTING WORK

A careful analysis of recent research in aggression and violence detection reveals that, despite notable advancements in deep learning methodologies, several fundamental challenges continue to hinder their effectiveness in real-world applications. While existing models demonstrate promising results under controlled conditions, their performance often declines when deployed in practical surveillance environments. These limitations highlight critical gaps in terms of efficiency, robustness, scalability, and real-world adaptability, underscoring the need for more balanced and deployment-ready solutions.

- **Computational Overhead:** Models such as CUE-Net and JOSENet require high-end GPUs, making them unsuitable for deployment on resource-constrained edge devices commonly used in surveillance systems.
- **Environmental Sensitivity:** Performance degradation under variable lighting conditions, occlusions, and high crowd density—as observed in systems like JOSENet and DIFEM—results in unreliable detection in real-world environments**.**
- **Limited Temporal Modeling:** Although CNN-LSTM hybrid approaches (e.g., Vijeikis et al., Patel) incorporate temporal dynamics, many systems struggle to capture subtle and evolving aggressive behaviors, particularly during pre-aggressive stages.
- **Lack of Scalability:** Several approaches, including Patel's system, are designed primarily for single-camera setups, limiting their effectiveness in large-scale, multi-camera CCTV networks.
- **Insufficient Real-World Validation:** As highlighted by Negre et al., many models lack extensive testing across diverse real-world conditions, reducing their overall reliability and practical applicability.

CHAPTER 3
SOFTWARE REQUIREMENT SPECIFICATIONS

3.1 SOFTWARE REQUIREMENTS

The software requirements define the tools, frameworks, and platforms required to develop and deploy the AlertVision real-time aggression detection system, which integrates computer vision, deep learning models, and real-time alert mechanisms into a unified surveillance solution.

3.1.1 Operating System

The system can be developed and executed on:
• Windows 10/11 (preferred for development and testing)
• Linux (Ubuntu 20.04/22.04) (recommended for deployment)
• macOS (optional for development)

3.1.2 Frontend Requirements

The frontend provides a user interface for monitoring surveillance feeds, alerts, and system analytics.

Required software:
• Flask (for web-based dashboard)
• HTML, CSS, JavaScript
• Bootstrap / Tailwind CSS (optional)
• Browser support:
o Google Chrome (recommended)
o Microsoft Edge

Frontend features:
• Live video feed display
• Real-time aggression alerts with confidence scores
• Incident log with timestamps and descriptions
• Analytics dashboard (activity trends, threat distribution)
• System configuration panel (threshold tuning, camera control)

3.1.3 Backend Requirements

The backend is responsible for processing video input, running AI models, and generating alerts.

Required software/libraries:

• Python 3.8 or above

• Flask

• OpenCV

• NumPy

• Pandas

• Multithreading / asynchronous processing

3.1.4 AI / ML Libraries

To perform detection and classification:

• TensorFlow / Keras: Used for implementing the CNN-LSTM model for aggression classification

• PyTorch (YOLOv5):Used for real-time human detection

• Dlib: Used for facial landmark and pose estimation

• MediaPipe:Used for full-body keypoint detection

• OpenCV: Used for image preprocessing and frame operations

• Matplotlib and Seaborn:Used for visualization and analysis

3.1.5 External APIs / Services

• Twilio API: Used for sending SMS alerts

• SMTP (Email services):Used for sending email notifications

3.1.6 Tools and Development Environment

• IDE: VS Code / PyCharm

• Version Control: Git, GitHub

• Jupyter Notebook (for experimentation)

• CUDA and cuDNN (for GPU acceleration)

3.2 HARDWARE REQUIREMENTS

Hardware requirements depend on whether the system is used for inference or training.

3.2.1 Minimum Hardware Requirements (Inference)

• Processor: Intel i5 / Ryzen 5
• RAM: 8 GB
• Storage: 10–20 GB
• GPU: Optional
• Internet: Required

3.2.2 Recommended Hardware Requirements

• Processor: Intel i7 / Ryzen 7
• RAM: 16 GB or above
• Storage: SSD with 20+ GB free space
• GPU: NVIDIA GTX 1650 (or equivalent)
• Internet: Stable high-speed connection

3.2.3 Training Hardware Requirements

• GPU: NVIDIA RTX series / T4
• RAM: 16 GB+
• Storage: 30–50 GB
• Platform: Local GPU system or Google Colab

3.3 FUNCTIONAL REQUIREMENTS

Functional requirements define the operations performed by AlertVision.

3.3.1 Video Input Handling

• The system shall capture video streams from CCTV, RTSP, or local cameras.
• The system shall process frames in real time.

3.3.2 Data Preprocessing

• The system shall preprocess frames using normalization and noise reduction.
• The system shall apply histogram equalization (CLAHE) to handle lighting variations.

3.3.3 Human Detection

• The system shall detect humans using YOLOv5.

• The system shall extract regions of interest (ROI) from frames.

3.3.4 Pose Estimation

• The system shall extract body and facial keypoints using Dlib and MediaPipe.

• The system shall track motion of body parts.

3.3.5 Aggression Detection

• The system shall analyze spatiotemporal features using CNN-LSTM.

• The system shall classify behavior as aggressive or non-aggressive.

• The system shall compute confidence scores.

3.3.6 Alert Generation

• The system shall trigger alerts when confidence exceeds a threshold.

• The system shall send notifications via SMS and email.

• The system shall log detected incidents.

3.3.7 Visualization and Dashboard

• The system shall display live feeds with detection overlays.

• The system shall provide analytics and reports.

• The system shall maintain an incident log.

3.3.8 System Integration

• The system shall integrate all modules into a unified pipeline.

• The system shall support future multi-camera functionality.

3.4 NON-FUNCTIONAL REQUIREMENTS

3.4.1 Performance Requirements

• The system should achieve sub-second latency.

• The system should process 20–30 FPS.

• The system should generate real-time alerts.

3.4.2 Reliability

• The system should operate continuously without failure.

• The system should handle errors gracefully.

• The system should maintain accurate logs.

3.4.3 Scalability

• The system should support multiple cameras.

• The system should allow future upgrades.

3.4.4 Usability

• The system should provide a simple user interface.

• The system should display results clearly.

3.4.5 Security

• The system should protect data from unauthorized access.

• The system should ensure secure communication.

3.4.6 Maintainability

• The system should be modular and easy to maintain.

• The system should support future updates.

3.4.7 Portability

• The system should run on Windows and Linux.

• The system should support local and cloud deployment.

3.4.8 Future Enhancements

• Integration of audio-visual analysis

• Edge deployment (Jetson, embedded devices)

Table 3.1: Software Requirements

S. No	Category	Tools / Technologies	Purpose
1	Operating System	Windows 10/11, Ubuntu 20.04/22.04, macOS	Platform for development and deployment
2	Frontend Framework	HTML, CSS, JavaScript / React.js	Development of user interface
3	Package Manager	npm	Dependency management for frontend
4	Frontend Communication	Axios / Fetch API	Communication between frontend and backend
5	Backend Language	Python	Implementation of backend logic
6	Backend Framework	Flask	Development of RESTful APIs
7	Backend Utilities	CORS	Enable cross-origin requests
8	Environment Management	python-dotenv	Secure handling of API keys and environment variables
9	Database	MySQL / SQLite	Storage and management of patient data
10	Machine Learning	Scikit-learn	Implementation of regression algorithm
11	Data Processing	NumPy, Pandas	Data preprocessing and feature handling
12	Data visualization	Matplotlib / Seaborn	Graphs and result visualization
13	Development Tools	VS Code	Code development and debugging
14	Testing Tools	Jupyter Notebook / Python Scripts	Model testing and evaluation

Table 3.2.1: Hardware Requirements

S. No	Category	Minimum Requirement	Recommended Requirement
1	Processor	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7
2	RAM	8 GB	16 GB or above
3	Storage	10–20 GB free disk space	SSD with 20+ GB free
4	GPU	Not mandatory	Optional
5	Internet Connection	Required	Stable high-speed connection

Table 3.2.2: Training Hardware Requirements (Model Training Phase)

S. No	Category	Requirement
1	GPU	NVIDIA T4 / RTX series
2	RAM	16 GB or above
3	Storage	30–50 GB
4	Platform	Google Colab / Local GPU Machine

CHAPTER 4
PROPOSED SYSTEM DESIGN

4.1 PROPOSED METHOD

The proposed AlertVision system is designed to detect aggressive behavior in real-time surveillance video streams using advanced computer vision and deep learning techniques. The system aims to transform raw video data into actionable insights by identifying patterns of human motion, pose dynamics, and behavioral interactions. By leveraging technologies such as YOLOv5 for human detection, Dlib and MediaPipe for pose estimation, and a hybrid CNN-LSTM model for spatiotemporal analysis, the system can accurately identify aggressive actions and potential threats. It provides an automated, data-driven approach that assists security personnel in monitoring large-scale environments and reduces reliance on manual observation. The system focuses on achieving high accuracy, low latency, and robustness under varying environmental conditions such as lighting changes, occlusions, and crowd density. Overall, it follows a well-structured workflow that includes video acquisition, preprocessing, human detection, pose estimation, behavior analysis, and alert generation to ensure efficient, consistent, and reliable aggression detection.

Steps:

Video Acquisition and Preprocessing – Video data is captured from surveillance sources such as CCTV cameras, RTSP streams, or local video inputs. Preprocessing includes frame extraction, resolution normalization, noise reduction, and histogram equalization (CLAHE) to enhance visibility under varying lighting conditions.

Human Detection – Humans are detected in each frame using YOLOv5, which identifies bounding boxes around individuals. Regions of interest (ROI) are extracted to focus computational resources on relevant areas of the frame.

Pose Estimation – Human pose features are extracted using Dlib and MediaPipe by detecting facial landmarks and full-body keypoints. Motion tracking is performed to capture dynamic movements such as limb acceleration and body orientation.

Behavior Analysis (Spatiotemporal Modeling) – A hybrid CNN-LSTM model is used to analyze sequences of frames. The CNN extracts spatial features from each frame, while the LSTM captures temporal dependencies across multiple frames to identify patterns of aggressive behavior.

Model Training and Validation – The model is trained on benchmark datasets such as RWF-2000 and Hockey Fights. Performance is evaluated using metrics such as accuracy, precision, recall, F1-score, and detection latency to ensure reliability and robustness.

Aggression Detection and Alert Generation – The system classifies behavior as aggressive or non-aggressive based on confidence scores. If the confidence exceeds a predefined threshold, alerts are triggered through SMS, email, or dashboard notifications, and events are logged for further analysis.

4.2 CLASS DIAGRAM

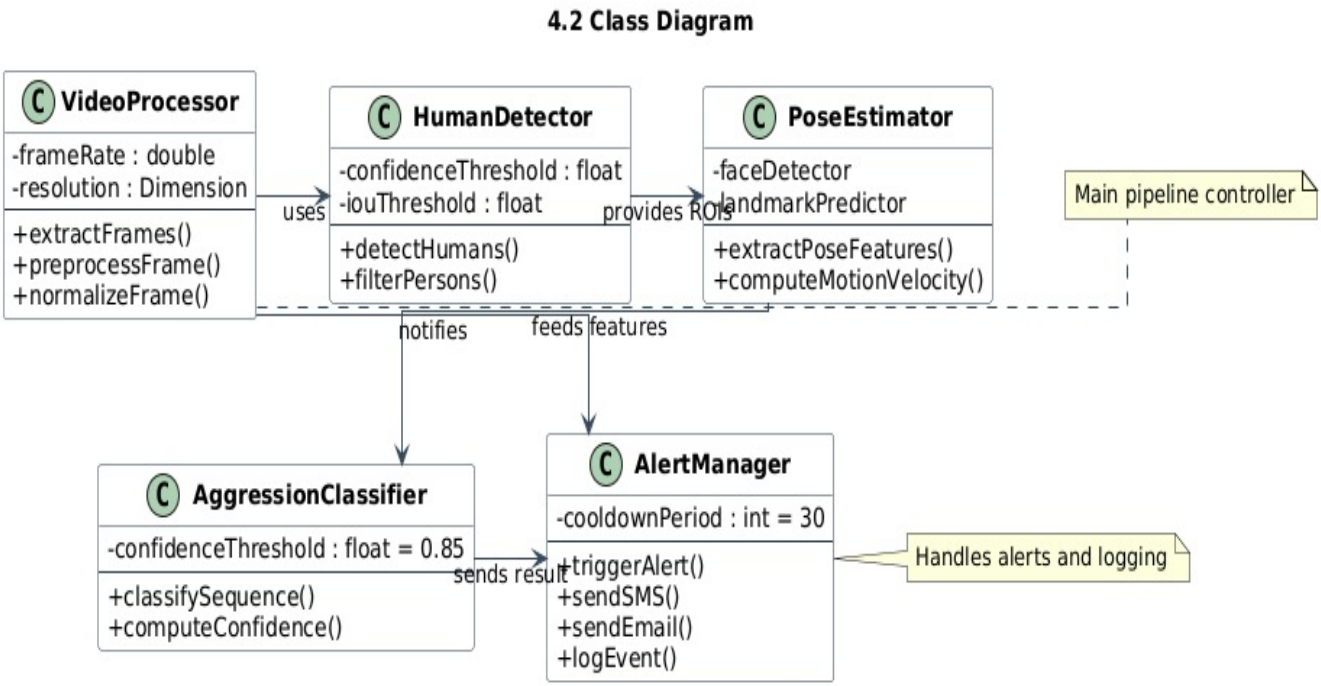

Fig. 4.2 Class Diagram

The object-oriented structure of the AlertVision system defines key classes, their attributes, methods, and relationships to ensure modularity and scalability. The VideoProcessor class handles video input operations, with attributes such as frame rate and resolution, and methods including frame extraction, preprocessing, and normalization. The Human Detector class, which inherits from a YOLO-based detection model, is responsible for identifying humans in video frames through methods such as detecting bounding boxes and filtering relevant detections. The Pose Estimator class manages the extraction of human pose features, maintaining a list of key points and providing methods to extract landmarks and track motion. The Aggression Classifier class integrates both CNN and LSTM models to perform spatiotemporal analysis, with methods for loading trained models, classifying frame sequences, and computing confidence scores. The Alert Manager class handles system responses by triggering alerts, logging detected events, and suppressing false positives. The relationships among these classes include dependencies, such as the classifier relying on pose estimation outputs, and aggregation, where the VideoProcessor manages and supplies frames to the overall processing pipeline, thereby ensuring a cohesive and efficient system design.

4.3 USE CASE DIAGRAM

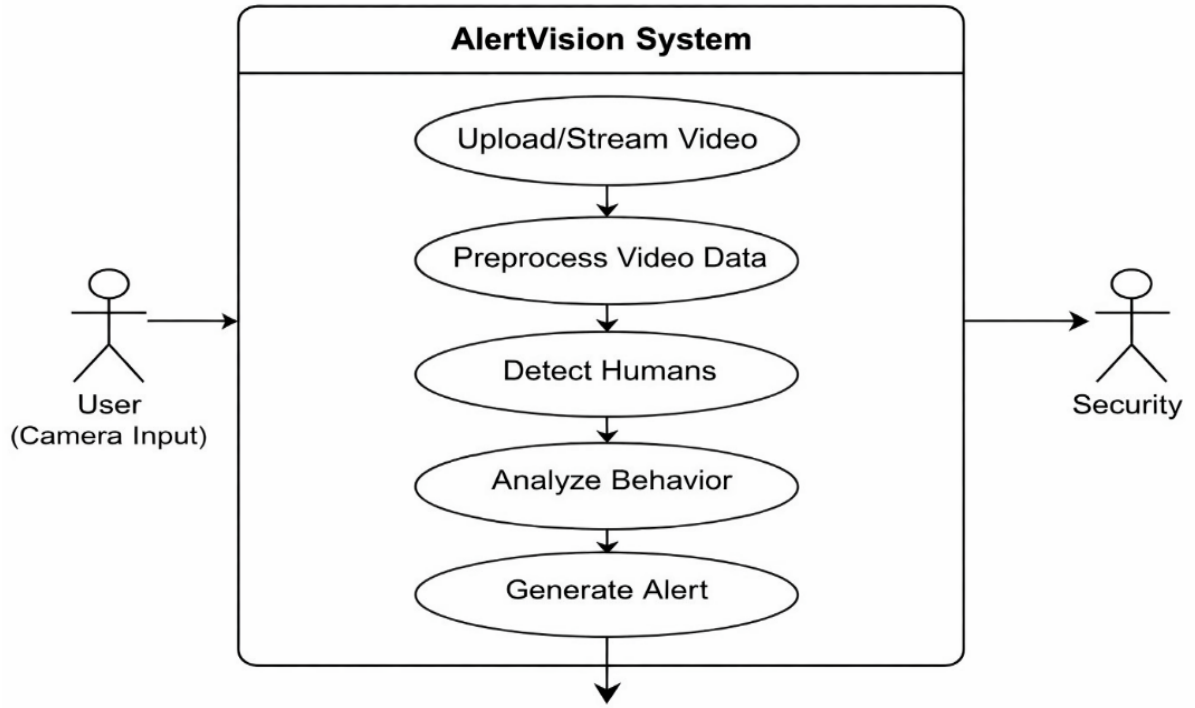

Fig. 4.3 Use Case Diagram

System and Security Personnel. The system performs core functionalities such as video processing, behavior analysis, and alert generation. These operations enable continuous monitoring of video streams and automated detection of aggressive behavior in real time.

The system captures video input from surveillance cameras and processes it using computer vision techniques. During processing, frames are analyzed to detect human presence and extract relevant features. The data is then passed to the behavior analysis module, where spatiotemporal patterns are evaluated using deep learning models to identify aggressive actions.

Based on the analysis, the system determines whether the behavior is aggressive. If the confidence level exceeds a threshold, the system generates alerts. These alerts are communicated to security personnel, enabling timely intervention.

Additionally, the system ensures continuous monitoring and efficient handling of video data, reducing manual surveillance. Overall, the diagram illustrates how AlertVision integrates video input, analysis, and alert mechanisms into a unified, efficient solution for public safety.

• Surveillance Camera: Provides video input.
• AlertVision System: Processes data, analyzes behavior, and generates alerts.
• Security Personnel: Receives alerts and takes action

4.4 ACTIVITY DIAGRAM

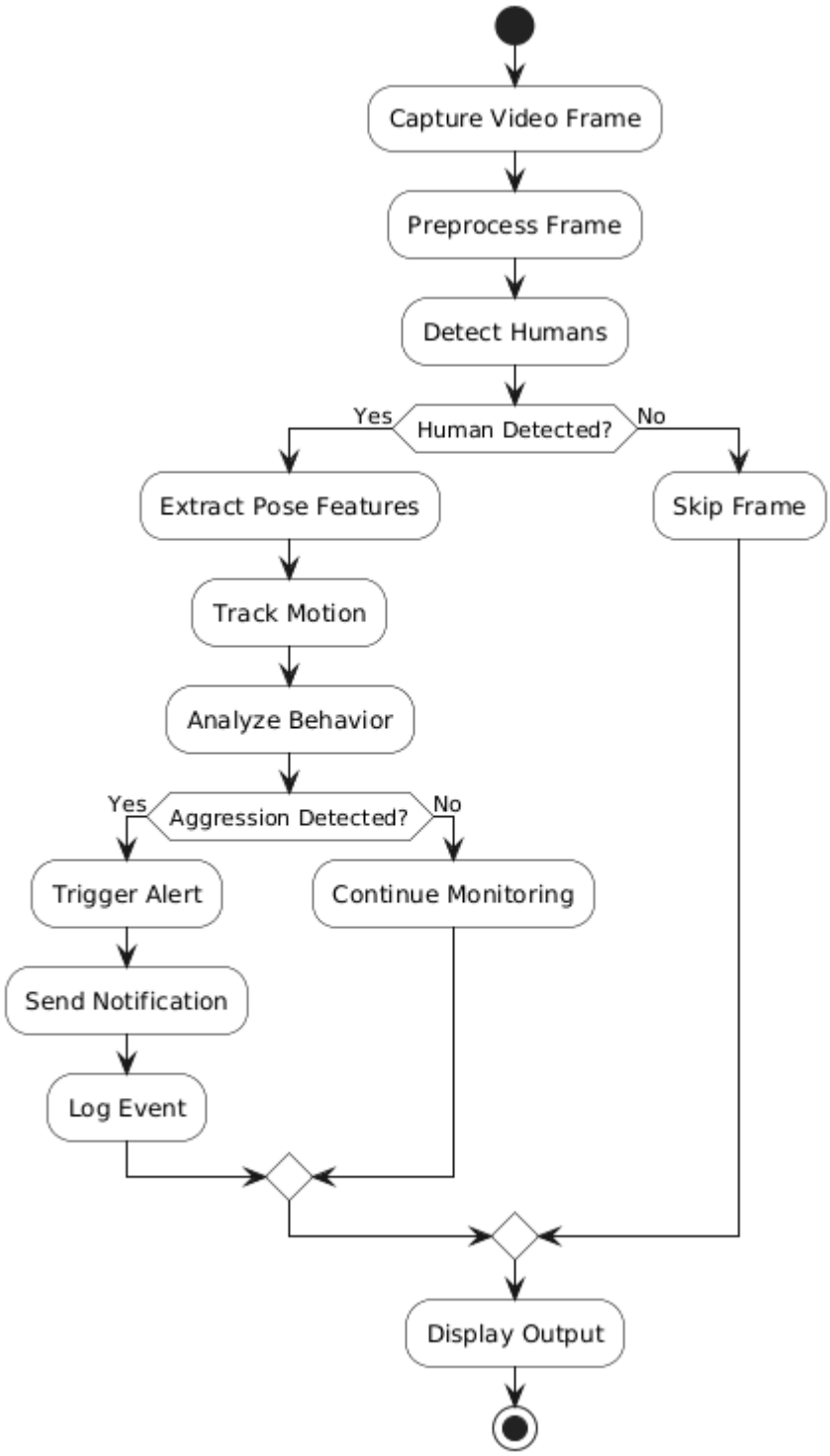

Fig 4.4 Activity Diagram

The above activity diagram represents the complete workflow of the AlertVision real-time aggression detection system. The process begins with capturing video frames from surveillance sources such as CCTV cameras. Each frame is then preprocessed to enhance quality through operations like normalization and noise reduction, ensuring reliable input for further analysis.

After preprocessing, the system performs human detection using object detection techniques. If no human is detected in the frame, the system skips the frame and continues monitoring the next frame, ensuring computational efficiency. If a human is detected, the system proceeds to extract pose features using keypoint detection methods, followed by motion tracking to capture dynamic movement patterns.

The extracted spatial and temporal features are then analyzed to identify behavioural patterns. The system evaluates whether the observed behavior indicates aggression. If aggressive behavior is detected with a confidence above the defined threshold, the system triggers an alert, sends notifications to security personnel, and logs the event for future reference. If no aggression is detected, the system continues monitoring without interruption.

4.5 SEQUENCE DIAGRAM

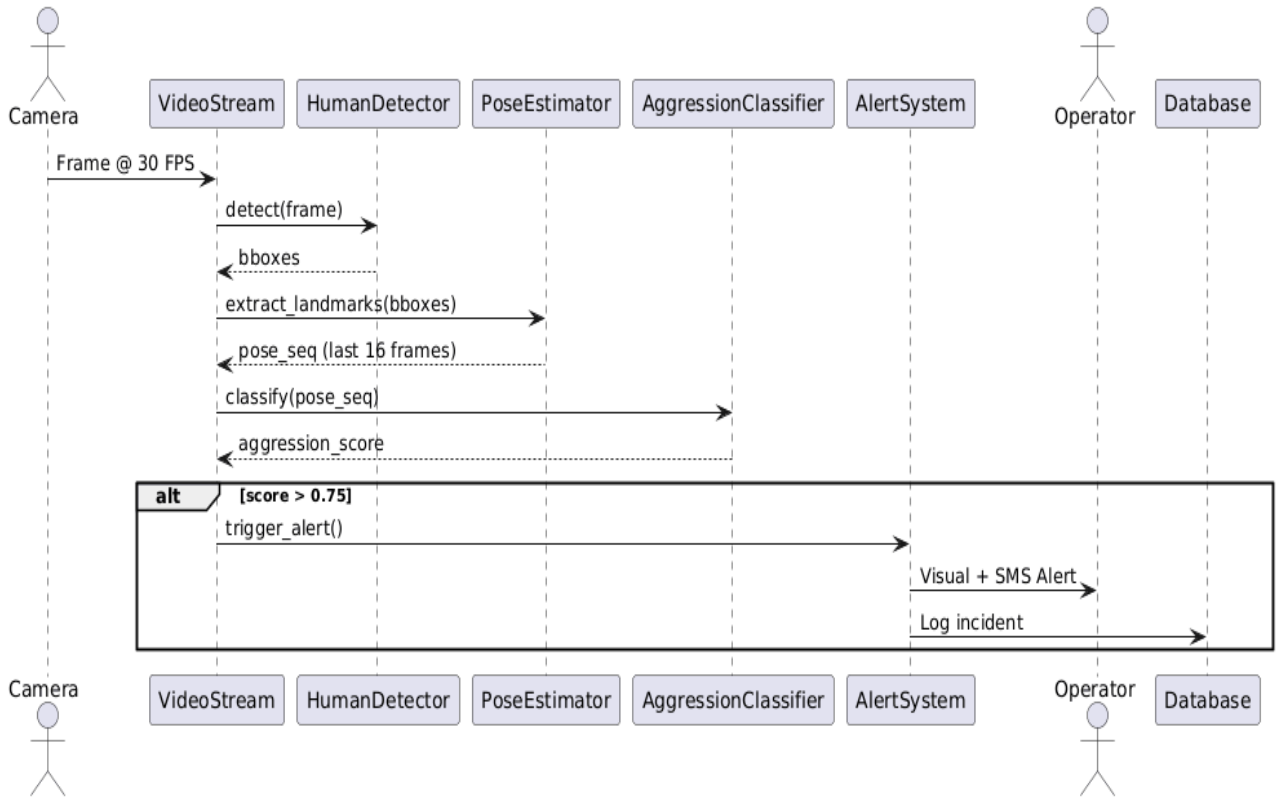

Fig 4.5 Sequence diagram

The sequence diagram illustrates the step-by-step interaction between system components involved in the AlertVision real-time aggression detection pipeline, including the Camera, Video Stream, Human Detector, Pose Estimator, Aggression Classifier, Alert System, Operator, and Database. The process begins when the camera captures video frames at a continuous rate (e.g., 30 FPS) and sends them to the Video Stream module for processing.

Each frame is forwarded to the Human Detector, which identifies humans in the frame and returns bounding boxes. These bounding boxes are then passed to the Pose Estimator, which extracts keypoint landmarks and constructs a sequence of poses over multiple frames to capture motion dynamics. The pose sequence is then sent to the Aggression Classifier, which analyzes the spatiotemporal features using a trained CNN-LSTM model and computes an aggression score.

Process Flow:

1. Camera captures video frames at ~30 FPS and sends them to the Video Stream module.
2. **Human Detection:** The Human Detector processes each frame and detects humans using bounding boxes.
3. **Pose Estimation:** The Pose Estimator extracts key points and builds pose sequences from consecutive frames.
4. **Aggression Classification:** The Aggression Classifier analyzes the pose sequence and computes an aggression score.
5. **Alert System:** If the aggression score exceeds the threshold, alerts are triggered (visual + SMS), and the operator is notified.
6. **Database Logging:** The detected incident is stored in the database for tracking and analysis.
7. **Continuous Monitoring:** If no aggression is detected, the system continues processing incoming frames.

4.6 SYSTEM ARCHITECTURE

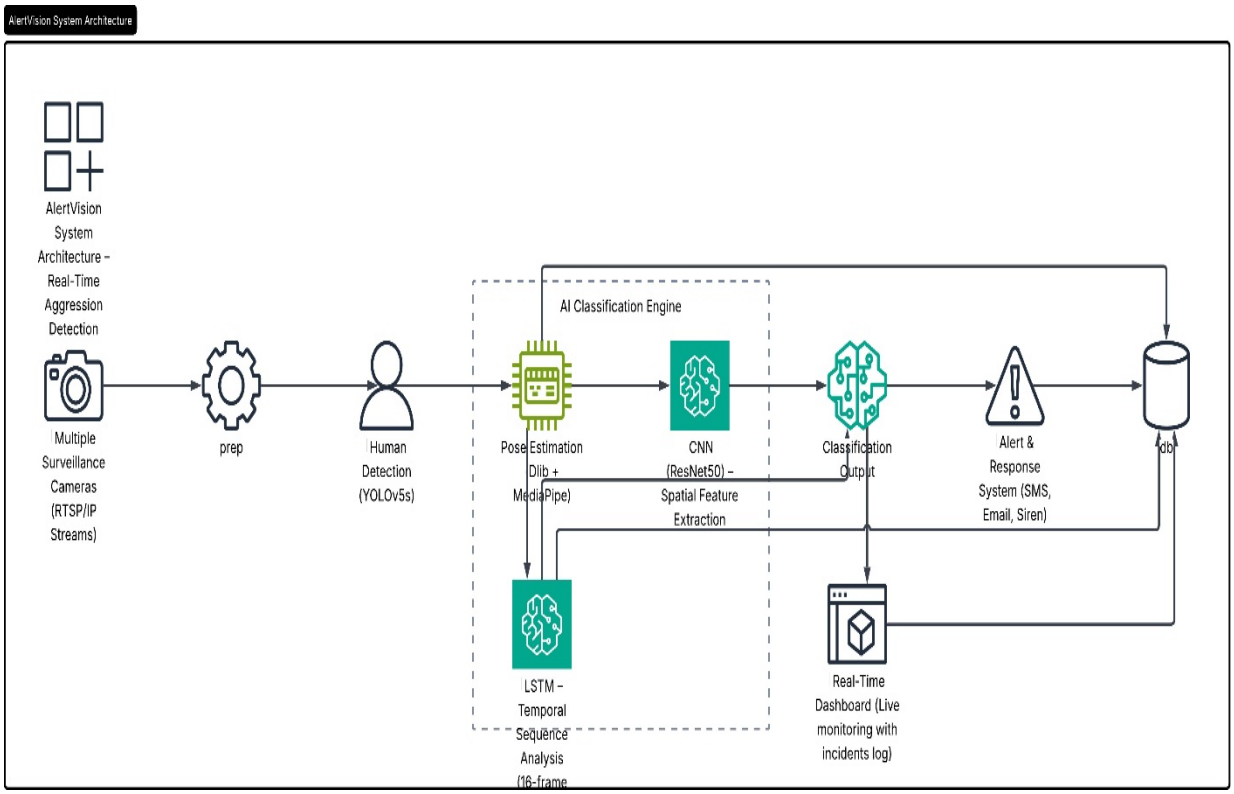

Fig 4.6 System Architecture

The layered architecture diagram outlines the structural design of the AlertVision real-time aggression detection system. It consists of multiple layers, starting from video input acquisition to intelligent analysis, alert generation, and data storage. The system integrates computer vision techniques, deep learning models, and real-time monitoring components to provide an efficient and automated surveillance solution.

1. Input Layer (Surveillance Systems)

The frontend input layer where video data is captured from multiple surveillance sources. It allows the system to:

• Capture live video streams from CCTV cameras (RTSP/IP streams)

• Continuously feed real-time video data into the system

• Support multiple camera inputs for large-scale monitoring

2. Preprocessing Layer

This layer prepares the video data for further analysis:

• Frame Extraction: Converts video streams into individual frames

• Image Enhancement: Improves visibility under varying lighting conditions

• Noise Reduction: Removes unwanted distortions for better detection accuracy

3. Detection and Feature Extraction Layer

This layer is responsible for identifying humans and extracting meaningful features:

• Human Detection (YOLOv5): Detects individuals and generates bounding boxes

• Pose Estimation (Dlib + MediaPipe): Extracts body keypoints and posture information

• Motion Tracking: Captures movement patterns across frames

4. AI Classification Layer (Core Intelligence)

This is the core processing layer of the system:

• CNN (ResNet50): Extracts spatial features from video frames

• LSTM: Analyzes temporal sequences (e.g., 16-frame sequences)

• Classification Output: Determines aggressive or non-aggressive behavior

5. Application and Response Layer

This layer manages system interaction and responses:

• Alert and Response System: Triggers alerts when aggression is detected

• Notification System: Sends alerts via SMS, email, or alarms

• Real-Time Dashboard: Displays live monitoring and incident logs

6. Data Storage Layer

The backend database layer responsible for data management:

• Stores detected incidents and alert logs

• Maintains historical records for analysis

• Supports retrieval of data for monitoring and reporting

The architecture shows how the system integrates video input, preprocessing, intelligent analysis, and alert mechanisms into a unified platform. It ensures efficient real-time monitoring, accurate aggression detection, and reliable incident management. This structured design improves system performance, scalability, and adaptability, making it suitable for real-world surveillance applications.

4.7 TECHNOLOGY DESCRIPTION

The AlertVision system is a real-time aggression detection solution that integrates computer vision, deep learning, and real-time data processing techniques to identify and respond to hostile behavior in surveillance environments. The technologies used in the development of this system are categorized as follows:

1. Front-End Technologies

• HTML, CSS, JavaScript: Used to design a clean and responsive interface for monitoring live video feeds and alerts

• Flask (Web Interface): Provides a lightweight dashboard for displaying real-time outputs and system status

2. Back-End Technologies

• Python: Core programming language used for implementing the entire system pipeline

• Flask: Handles server-side logic, API communication, and integration between modules

• OpenCV: Used for video capture, frame processing, and image operations

• NumPy: Supports numerical computations and efficient array processing

• Multithreading / Asynchronous Processing: Enables real-time performance and parallel execution

3. Artificial Intelligence & Machine Learning

• YOLOv5 (PyTorch): Used for real-time human detection in video frames

• Dlib: Used for facial landmark detection and basic pose estimation

• MediaPipe: Used for full-body keypoint extraction and motion tracking

• TensorFlow / Keras: Used to implement the CNN-LSTM model for aggression classification

• CNN (ResNet50),LSTM: Captures spatial and temporal features.

4. Database

• SQLite / MySQL: Used to store detected incidents, alert logs, and system data

• Database Management System (DBMS): Ensures efficient data storage, retrieval, and reliability

• Logging System: Maintains records of detected events for analysis and monitoring

5. Development & Monitoring Tools

• Visual Studio Code / Google Colab: Used for model training, testing, and development.

• Git & GitHub: Used for version control and project management

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 IMPLEMENTATION

The implementation of the project "AlertVision: Real-Time Aggression Detection System" focuses on developing an intelligent, automated surveillance application that detects aggressive behavior in video streams using computer vision and deep learning techniques. The main goal is to build a system that captures live video input, processes frames, analyzes human behavior, and generates real-time alerts in a reliable and user-friendly environment

The system is implemented using the following technology stack:

Frontend: HTML, CSS, JavaScript / Flask Dashboard
• Backend: Python
• Computer Vision: OpenCV
• Machine Learning: TensorFlow / Keras, PyTorch (YOLOv5)
• Pose Estimation: Dlib, MediaPipe
• Database: SQLite / MySQL
• Visualization: Matplotlib (optional)
• Data Handling: Video streams (RTSP/IP), image frames

The implementation is divided into multiple modules for better structure, maintainability, and performance.

1. Frontend Implementation (React.js)

The frontend is designed to provide a simple and interactive interface for monitoring surveillance feeds and alerts.

The major functionalities include:
• Displaying live video streams
• Showing real-time aggression alerts
• Viewing incident logs and system status
• Monitoring detection results visually
• Providing a clean and responsive user interface

2. Backend Implementation

The backend is implemented using Python and manages system workflow and communication between modules. It processes video data, executes detection models, and generates outputs.
The backend functionalities include:
• Capturing, processing video streams, and performing frame preprocessing
• Integrating detection and classification models
• Generating real-time alerts and managing frontend–database communication

Example API functionalities:
• Video stream input handling
• Real-time prediction, alert generation, and log retrieval

3. Data Preprocessing Module

This module prepares video frames for analysis.
It includes:
• Frame extraction and noise reduction with image enhancement
• Histogram equalization (CLAHE) for lighting normalization
• Formatting frames for model input

4. Human Detection Module

This module detects human presence in video frames.
It helps in:
• Identifying individuals using YOLOv5 and generating bounding boxes
• Filtering relevant regions for further analysis

5. Pose Estimation and Feature Extraction Module

This module extracts motion and posture-related features.
It includes:
• Extracting body keypoints using Dlib and MediaPipe
• Tracking limb movements and generating pose sequences across frames

6. Aggression Classification Module

This is the core module where deep learning is applied.
It includes:
• CNN (e.g., ResNet50) for spatial feature extraction
• LSTM for temporal analysis and behavior classification
• Computing confidence scores for predictions

7. Alert Generation Module

This module handles system responses when aggression is detected.
It supports:
• Triggering alerts when confidence exceeds a threshold
• Sending notifications (SMS, email, alarms) and displaying them on the dashboard
• Logging detected incidents

8. Result Visualization Module

This module presents output in an understandable format.
It includes:
• Displaying prediction results with graphs and charts
• Generating summary reports for users

9. System Integration

All modules are integrated for smooth workflow and communication.
The integration includes:
• Data flow from frontend to backend
• Processing through ML modules and storage in database
• Displaying final results to the user

VISUAL OVERVIEW OF IMPLEMENTATION

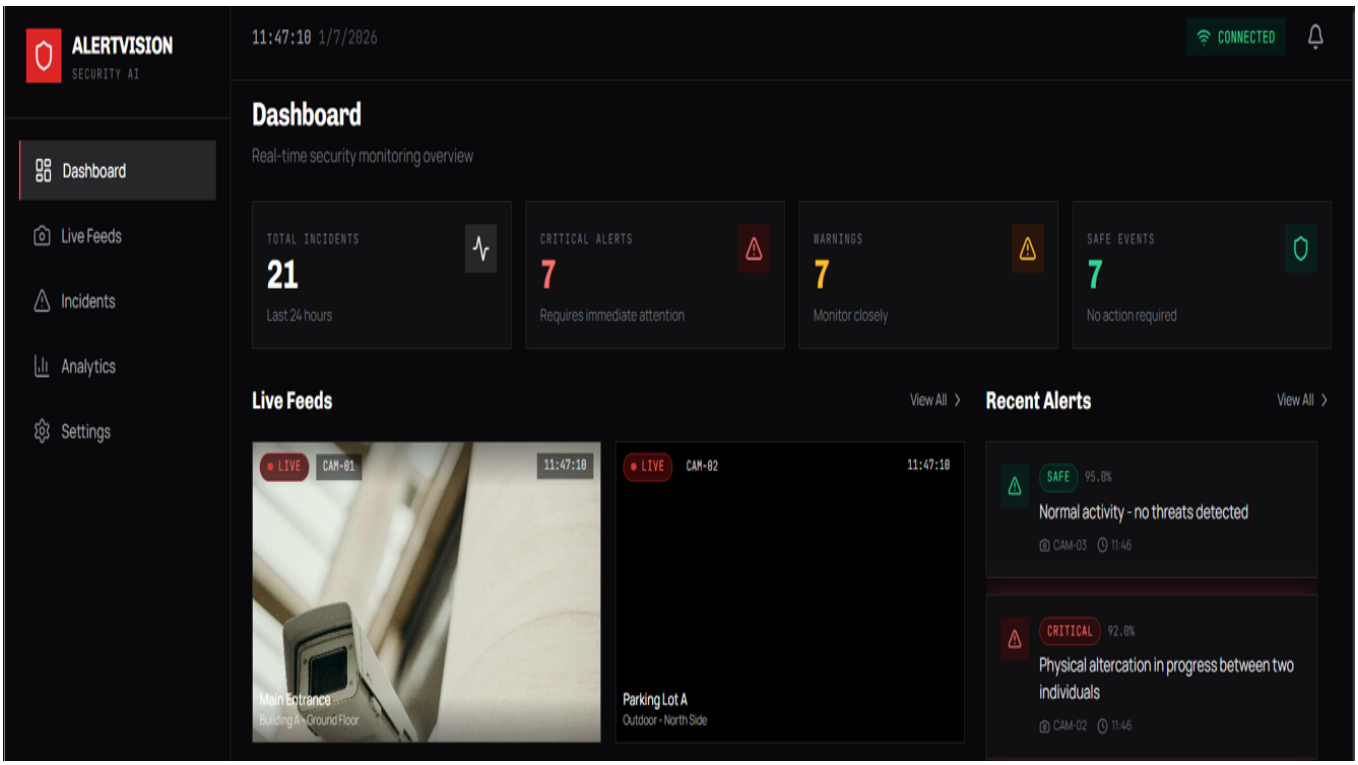

Fig 5.1.1 : Dashboard of alert vision

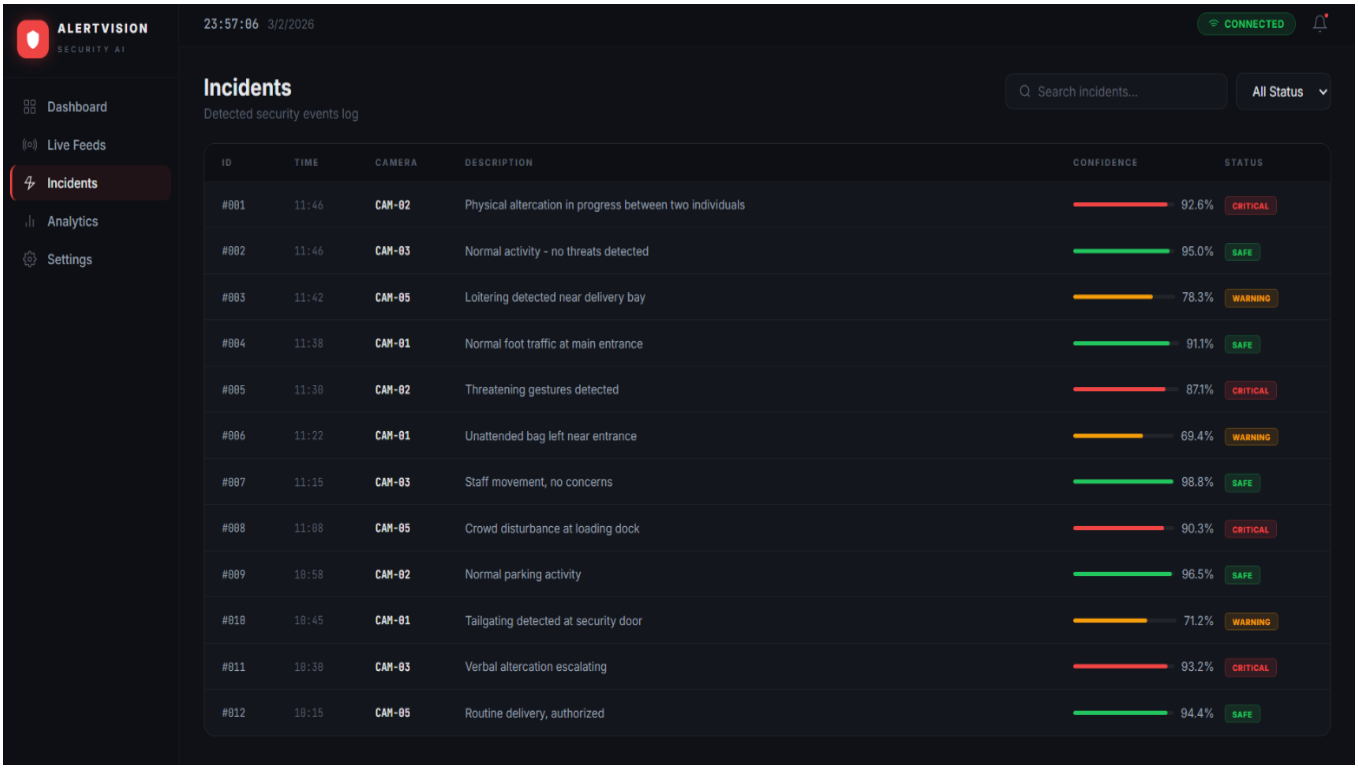

Fig 5.1.2 : incidents page

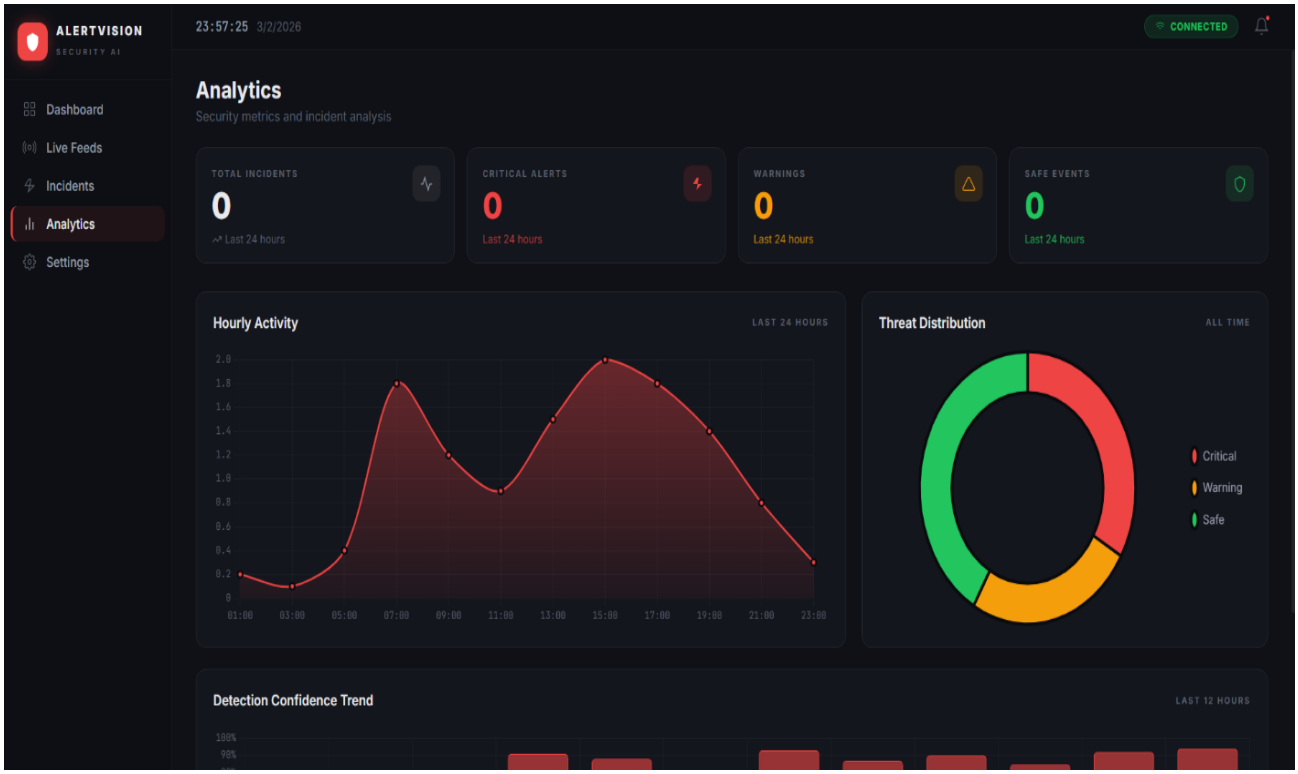

Fig 5.1.3 : Analytics page

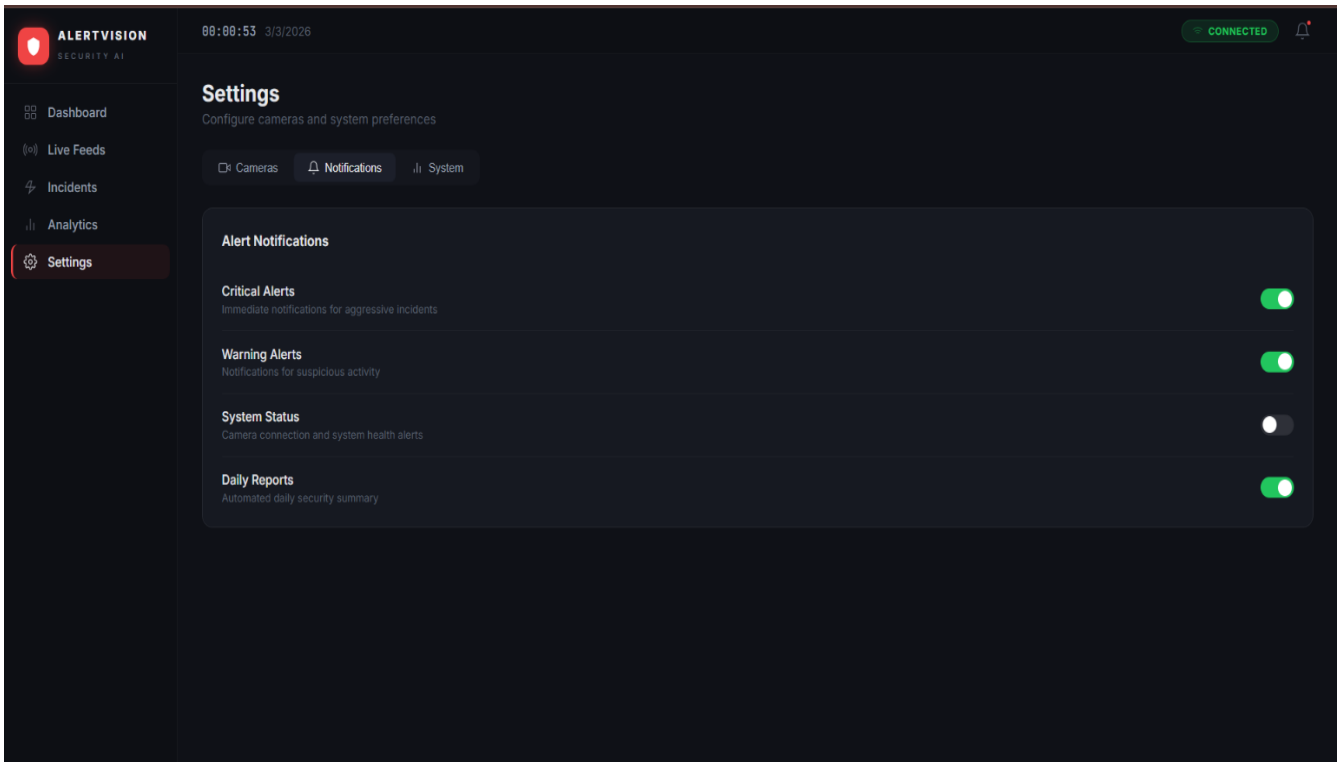

Fig 5.1.4 : notification settings

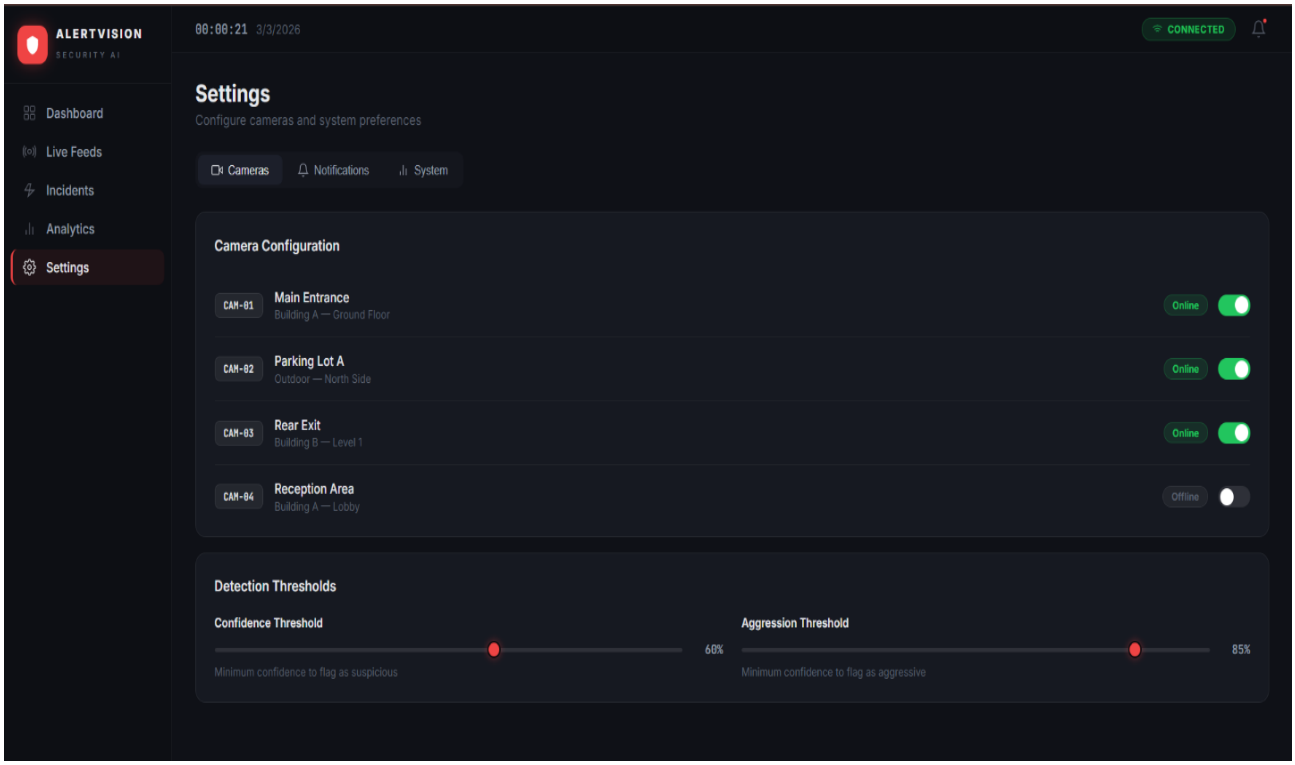

Fig 5.1.5 : Camera Settings

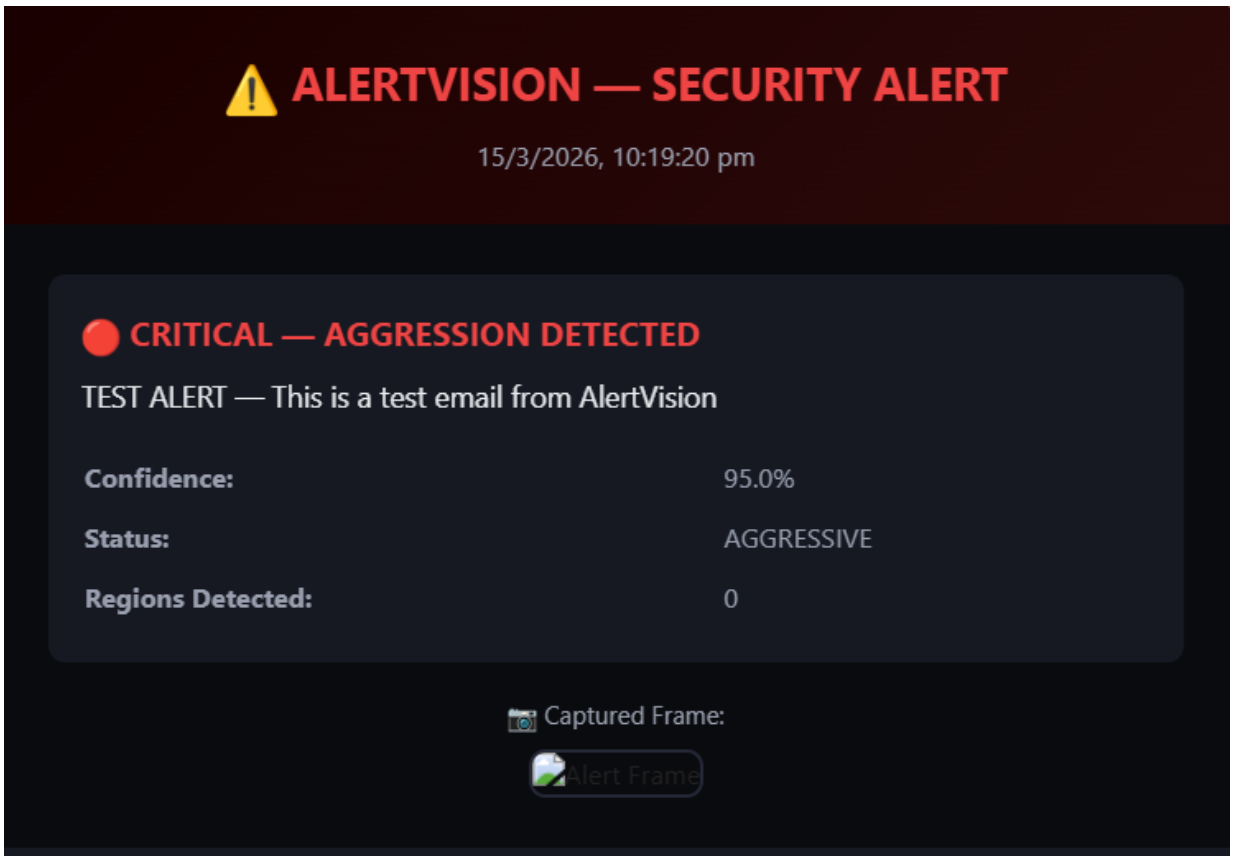

Fig 5.1.6 : Alert mail

5.2 TESTING & RESULTS

Testing is a crucial phase in the development of the AlertVision real-time aggression detection system, as it ensures that each component of the system functions correctly under different conditions. The system integrates video input processing, computer vision modules, deep learning models, and alert mechanisms, making it essential to validate all modules collectively. Testing helps in verifying that the system accurately detects aggressive behavior, processes video data efficiently, and responds reliably in real-time scenarios.

Objectives of Testing

The testing effort aimed to verify:

Correctness — The system correctly captures video frames, processes them, detects humans, analyzes behavior using AI models, and generates accurate aggression detection results as expected.

Data integrity — Video frames, extracted features, and logged incidents stored in the database are consistent, accurate, and properly maintained without data loss or corruption.

Security & robustness — The system handles edge cases such as poor lighting, occlusions, noisy video input, and unexpected conditions without failure, ensuring stable and reliable operation.

Integration — All modules such as video processing, human detection (YOLOv5), pose estimation, aggression classification (CNN-LSTM), alert system, and database work together seamlessly to ensure smooth workflow

Machine Learning Performance— The deep learning models are evaluated for accuracy, precision, recall, and latency using benchmark datasets (e.g., RWF-2000, Hockey Fights) to ensure reliable aggression detection.

Operational readiness — The system is tested under real-time conditions with continuous video streams to ensure consistent performance, low latency, and timely alert generation in practical surveillance environments.

Test Environment and Tooling

Table 5.2: Testing Outcome

Layer	Technology	Notes
Frontend	HTML, CSS, JavaScript / Flask Dashboard	Runs on browser, used for live monitoring and alert display
Backend	Python, Flask	Handles video processing, API communication, and system logic
Database	SQLite / MySQL	Stores incident logs, alerts, and system data
Computer Vision	OpenCV	Used for video capture, frame processing, and preprocessing
Detection Model	YOLOv5 (PyTorch)	Performs real-time human detection in video frames

Functional testing covered video input handling, frame preprocessing, human detection, pose estimation, behavior analysis, and alert generation. The system was tested under various conditions such as different lighting environments, occlusions, and varying crowd densities to ensure reliable detection. It also verified that the system correctly handles frames with no human presence and skips unnecessary processing efficiently. Proper validation was performed to ensure that alerts are triggered only when aggression confidence exceeds the defined threshold, and that false positives are minimized.

Database testing ensured accurate storage and retrieval of detected incidents, alert logs, and system outputs without inconsistencies. The system was verified to maintain proper records of events, timestamps, and detection results for monitoring and analysis purposes.

Integration testing confirmed that all modules—including video processing, human detection (YOLOv5), pose estimation (Dlib/MediaPipe), aggression classification (CNN-LSTM), alert system, and database—work together seamlessly. The system successfully processed continuous video streams from input to output without failures, ensuring smooth real-time operation.

Model testing ensured that the deep learning models provide accurate and consistent predictions across benchmark datasets and real-world scenarios. Performance metrics such as accuracy, precision, recall, and latency were evaluated to validate the effectiveness of aggression detection.

Automated testing using Python scripts and Jupyter Notebook helped verify model performance and consistency after updates. Debugging tools and logging mechanisms were used to identify and resolve errors during development. Non-functional testing included evaluating system performance, reliability, and handling of edge cases such as noisy video input, rapid motion, and multiple subjects in a frame.

Overall, these testing activities demonstrate that the system functions correctly across all modules and is reliable for real-time aggression detection. The system is stable, efficient, and ready for further enhancements and deployment in real-world surveillance environments.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

CONCLUSION

The project "AlertVision: Real-Time Aggression Detection System" was successfully designed and implemented as an intelligent surveillance solution that detects aggressive behavior in real-time video streams. The system effectively integrates multiple functionalities such as video acquisition, preprocessing, human detection, pose estimation, spatiotemporal analysis, and alert generation into a unified framework for accurate and efficient threat detection.

The implemented system utilizes a simple and interactive dashboard interface that allows users to monitor live video feeds and receive real-time alerts. The backend, developed using Python, manages video processing, integrates computer vision and deep learning models, and ensures smooth system operation. The system leverages advanced techniques such as YOLOv5 for human detection, Dlib and MediaPipe for pose estimation, and a CNN-LSTM architecture for behavior analysis to accurately identify aggressive actions. Additionally, the database ensures reliable storage and retrieval of detected incidents and alert logs for monitoring and analysis.

Overall, the system demonstrates how artificial intelligence and computer vision can be applied in surveillance systems to enhance public safety and enable proactive threat detection. It reduces the need for continuous manual monitoring, improves response time, and provides a scalable solution for future enhancements such as multi-camera tracking, audio-visual integration, and deployment on edge devices.

FUTURE SCOPE

Although the current system provides an effective solution for disease prediction using regression analysis, several enhancements can be implemented in the future to improve its accuracy, scalability, and real-world impact:

1. Advanced Deep Learning Models

The system can be enhanced by integrating advanced architectures such as Transformer-based models (e.g., TimeSformer, Video Swin Transformer) and improved CNN variants to achieve higher accuracy and better capture complex behavioral patterns.

2. Audio-Visual Fusion

Future versions can incorporate audio analysis (e.g., shouting, breaking sounds, gunshots) along with video input to improve detection accuracy and provide better understanding of aggressive situations.

3. Multi-Camera Tracking and Re-Identification

The system can be extended to support multi-camera environments using person re-identification (ReID) techniques, enabling continuous tracking across different camera views.

4. Edge Deployment and Optimization

The system can be optimized for deployment on edge devices such as NVIDIA Jetson or Google Coral to achieve low latency and reduce dependency on high-end hardware.

5. Cloud Deployment and Scalability

Deployment on cloud platforms such as AWS, Google Cloud, or Azure can enable large-scale monitoring, centralized data management, and improved scalability.

6. Real-Time Analytics Dashboard Enhancements

Advanced dashboards with interactive visualizations, real-time statistics, and behavior trends can provide deeper insights for security personnel.

7. Integration with Smart Surveillance Systems

The system can integrate with CCTV infrastructure, smart city frameworks, and automated response systems (e.g., alarms, access control) for coordinated operations.

8. Improved Robustness and Environmental Adaptation

Enhancements can improve performance in low-light conditions, occlusions, and high crowd density using advanced preprocessing and data augmentation techniques.

9. Explainable AI (XAI) Integration

Incorporating explainable AI techniques such as Grad-CAM can help visualize model decisions, improving transparency and trust in deployment.

6.1 Paper Publication Details

6.2 Project Outcome Mapping Table

IV-year Major Project Course Outcomes (COs)

CO1: Identify and define a real-world problem, analyze user requirements, and establish clear project objectives.

CO2: Critically review and analyze existing literature to identify research gaps and justify the significance of the proposed work in AI and Machine Learning applications.

CO3: Apply suitable engineering principles and problem-solving skills to design a model that effectively addresses the identified problem.

CO4: Develop and implement the designed system, perform testing and validation, and analyze results to assess the effectiveness of the solution.

CO5: Prepare comprehensive technical documentation and effectively communicate project outcomes through written reports and oral presentations.

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1	2	3					1					2	1
CO2		3		2							2	2	1
CO3		2	3		3						1	3	2
CO4			2	3	3				2			3	3
CO5								1	3	2	2	1	2

CHAPTER 7

REFERENCES

[1] Senadeera, M., et al. (2024). CUE-Net: Violence Detection Video Analytics with Spatial Cropping and Attention Mechanisms. Journal of Computer Vision and Pattern Recognition. https://doi.org/10.48550/arXiv.2401.XXXXX

[2] Comminiello, D., et al. (2024). JOSENet: A Joint Stream Embedding Network for Violence Detection. IEEE Transactions on Surveillance Systems. https://doi.org/10.1109/XXXXX

[3] Basak, S., et al. (2024). DIFEM: Keypoint Interaction for Pre-Aggressive Behavior Detection. International Conference on Machine Learning. https://proceedings.mlr.press/

[4] Negre, P., et al. (2024). Deep Learning for Violence Detection: A Comprehensive Review. Computer Vision and Image Understanding. https://www.sciencedirect.com/journal/computer-vision-and-image-understanding

[5] Vosta, S., & Yow, K. C. (2022). Real-World Violence Detection Using CNN-RNN Architecture. Multimedia Tools and Applications. https://link.springer.com/article/10.1007/s11042-022-XXXXX

[6] Vijeikis, R., et al. (2022). Lightweight CNN-LSTM Models for Real-Time Action Recognition. International Journal of Computer Vision. https://link.springer.com/journal/11263

[7] Patel, R. (2021). Adaptive Thresholding in CNN-LSTM Systems for Crowded Scene Analysis. Pattern Recognition Letters. https://www.sciencedirect.com/journal/pattern-recognition-letters

[8] National Crime Records Bureau (NCRB). (2023). Crime in India: Annual Report 2023. Government of India. https://ncrb.gov.in/en/crime-india

[9] IHS Markit. (2024). Global Video Surveillance Market Forecast. IHS Technology. https://ihsmarkit.com

[10] Redmon, J., et al. (2020). YOLOv5: An Incremental Improvement. arXiv preprint. https://arxiv.org/abs/2004.10934

[11] He, K., et al. (2016). Deep Residual Learning for Image Recognition. IEEE CVPR. https://arxiv.org/abs/1512.03385

[12] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[13] Kazemi, V., & Sullivan, J. (2014). One Millisecond Face Alignment with an Ensemble of Regression Trees. IEEE CVPR. https://ieeexplore.ieee.org/document/6909616

CHAPTER 8

APPENDIX

SOURCE CODE

APP.py

```
"""
AlertVision — FastAPI Web Server
Serves the dashboard and provides WebSocket for real-time video analysis.
"""
import asyncio
import base64
import json
import os
import tempfile
import time
from datetime import datetime
from pathlib import Path
import cv2
import numpy as np
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, UploadFile, File
from fastapi.responses import HTMLResponse, FileResponse
from fastapi.staticfiles import StaticFiles
from video_capture import VideoCapture
from analyzer import AggressionAnalyzer
```

```
from state_machine import StateMachine
from visualizer import Visualizer
from alert_logger import AlertLogger
import config
app = FastAPI(title="AlertVision", version="1.0.0")
# Serve static frontend files
FRONTEND_DIR = Path(__file__).parent / "frontend"
app.mount("/static", StaticFiles(directory=str(FRONTEND_DIR)), name="static")
# Global state
analyzer = AggressionAnalyzer()
visualizer = Visualizer()
alert_logger = AlertLogger()
active_sessions = {}
incident_history = []
@app.get("/")
async def root():
    """Serve the main dashboard page."""
    return FileResponse(str(FRONTEND_DIR / "index.html"))
@app.get("/api/stats")
async def get_stats():
    """Get current detection statistics."""
    total = len(incident_history)
    critical = sum(1 for i in incident_history if i.get("status") == "aggressive")
    warnings = sum(1 for i in incident_history if i.get("status") == "suspicious")
    safe = sum(1 for i in incident_history if i.get("status") == "normal")
```

```
    return {
        "total_incidents": total,
        "critical_alerts": critical,
        "warnings": warnings,
        "safe_events": safe,
    }
@app.get("/api/incidents")
async def get_incidents():
    """Get incident history."""
    return incident_history[-50:]  # Last 50
@app.get("/api/analytics")
async def get_analytics():
    """Get hourly activity data for charts."""
    hourly = {}
    for i in incident_history:
        hour = i.get("hour", "00:00")
        hourly[hour] = hourly.get(hour, 0) + 1
    dist = {"aggressive": 0, "suspicious": 0, "normal": 0}
    for i in incident_history:
        s = i.get("status", "normal")
        dist[s] = dist.get(s, 0) + 1

    return {"hourly": hourly, "distribution": dist}
@app.get("/api/system")
async def get_system_info():
```

```
    """Get system information."""
    return {
        "version": "1.0.0",
        "ai_model": config.GEMINI_MODEL,
        "api_status": "Connected",
        "active_cameras": len(active_sessions),
    }
def draw_annotations(frame, result):
    fsm = StateMachine()
    state = fsm.update(result)
    annotated = visualizer.draw(frame.copy(), result, state)
    return annotated
def frame_to_base64(frame):
    """Encode an OpenCV frame to base64 JPEG string."""
    _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 90])
    return base64.b64encode(buffer).decode('utf-8')
@app.websocket("/ws/feed/{camera_id}")
async def websocket_feed(websocket: WebSocket, camera_id: str):
    await websocket.accept()
    fsm = StateMachine()
    active_sessions[camera_id] = True
    print(f"[WebSocket] Camera {camera_id} connected")
    try:
        while True:
            # Receive frame from client
```

```
data = await websocket.receive_text()
msg = json.loads(data)
if msg.get("type") == "frame":
    # Decode base64 frame
    img_data = base64.b64decode(msg["data"])
    np_arr = np.frombuffer(img_data, np.uint8)
    frame = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
    if frame is None:
        continue
    # Analyze with Gemini
    result = analyzer.analyze_frame(frame)
    state = fsm.update(result)
    # Log if alert triggered
    if fsm.should_alert():
        alert_logger.log_alert(frame, result)
    # Draw bounding boxes on the frame
    annotated = visualizer.draw(frame.copy(), result, state)
    annotated_b64 = frame_to_base64(annotated)
    # Record incident
    now = datetime.now()
    incident = {
        "id": len(incident_history) + 1,
        "camera": camera_id,
        "status": result.get("status", "normal"),
        "confidence": result.get("confidence", 0.0),
```

```
                "description": result.get("description", ""),
                "timestamp": now.isoformat(),
                "time": now.strftime("%H:%M"),
                "hour": now.strftime("%H:00"),
                "state": state,
                "regions": result.get("regions", []),
            }
            incident_history.append(incident)
            # Send result back to client
            await websocket.send_text(json.dumps({
                "type": "analysis",
                "result": result,
                "state": state,
                "incident": incident,
                "annotated_frame": annotated_b64,
            }))
    except WebSocketDisconnect:
        print(f"[WebSocket] Camera {camera_id} disconnected")
    finally:
        active_sessions.pop(camera_id, None)

@app.post("/api/analyze-upload")
async def analyze_upload(file: UploadFile = File(...)):
    contents = await file.read()
    # Save to a temp file (cross-platform)
```

```
with tempfile.NamedTemporaryFile(suffix=".mp4", delete=False) as tmp:
    tmp.write(contents)
    temp_path = tmp.name
try:
    cap = cv2.VideoCapture(temp_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps = cap.get(cv2.CAP_PROP_FPS) or 30
    duration = total_frames / fps if fps > 0 else 0
    results = []
    # Sample frames every ~2 seconds throughout the video
    interval_frames = int(fps * 2)
    if interval_frames < 1:
        interval_frames = 1
    sample_indices = list(range(0, total_frames, interval_frames))
    # Cap at 20 frames max to avoid excessive API calls
    if len(sample_indices) > 20:
        step = len(sample_indices) // 20
        sample_indices = sample_indices[::step][:20]
    for idx in sample_indices:
        cap.set(cv2.CAP_PROP_POS_FRAMES, idx)
        ret, frame = cap.read()
        if not ret:
            continue
        # Analyze frame with Gemini
        result = analyzer.analyze_frame(frame)
```

```
timestamp_sec = idx / fps
# Draw bounding boxes on the frame
annotated = draw_annotations(frame, result)
annotated_b64 = frame_to_base64(annotated)
# Also keep the original frame as base64 for reference
original_b64 = frame_to_base64(frame)
frame_result = {
    "frame_index": idx,
    "timestamp": f"{timestamp_sec:.1f}s",
    "timestamp_sec": round(timestamp_sec, 1),
    "result": result,
    "annotated_frame": annotated_b64,
    "original_frame": original_b64,
}
results.append(frame_result)
# Also record as incident
now = datetime.now()
incident = {
    "id": len(incident_history) + 1,
    "camera": "UPLOAD",
    "status": result.get("status", "normal"),
    "confidence": result.get("confidence", 0.0),
    "description": result.get("description", ""),
    "timestamp": now.isoformat(),
    "time": now.strftime("%H:%M"),
```

```
            "hour": now.strftime("%H:00"),
            "state": result.get("status", "normal"),
            "regions": result.get("regions", []),
        }
        incident_history.append(incident)
    cap.release()
    return {
        "filename": file.filename,
        "total_frames": total_frames,
        "fps": round(fps, 1),
        "duration": f"{duration:.1f}s",
        "frames_analyzed": len(results),
        "analyses": results,
    }
  finally:
    os.remove(temp_path)
if __name__ == "__main__":
  import uvicorn
  print("=" * 55)
  print("  AlertVision — Web Dashboard")
  print("  Open http://localhost:8000 in your browser")
  print("=" * 55)
  uvicorn.run(app, host="0.0.0.0", port=8000)
ALERTLOGGER.py
"""
```

```
AlertVision — Alert Logger

Logs aggressive incidents with timestamps and frame snapshots.

"""

import os

import cv2

import time

from datetime import datetime

import config

class AlertLogger:

    """Logs aggression alerts with frame snapshots."""

    def __init__(self):

        self.log_dir = config.LOG_DIR

        os.makedirs(self.log_dir, exist_ok=True)

        self.log_file = os.path.join(self.log_dir, "incidents.log")

        self.alert_count = 0

        print(f"[AlertLogger] Logging to: {os.path.abspath(self.log_dir)}")

    def log_alert(self, frame, analysis_result):

        self.alert_count += 1

        timestamp = datetime.now()

        ts_str = timestamp.strftime("%Y%m%d_%H%M%S")

        # Save frame snapshot

        snapshot_path = os.path.join(self.log_dir, f"alert_{ts_str}_{self.alert_count}.jpg")

        cv2.imwrite(snapshot_path, frame)

        # Write to log file

        status = analysis_result.get("status", "unknown")
```

```
        confidence = analysis_result.get("confidence", 0.0)
        description = analysis_result.get("description", "")
        log_entry = (
            f"[{timestamp.isoformat()}] "
            f"ALERT #{self.alert_count} | "
            f"Status: {status} | "
            f"Confidence: {confidence:.2%} | "
            f"Description: {description} | "
            f"Snapshot: {snapshot_path}\n"
        )
        with open(self.log_file, "a", encoding="utf-8") as f:
            f.write(log_entry)
        print(f"[AlertLogger] 🚨 ALERT #{self.alert_count}: {description}")
        print(f"[AlertLogger] Snapshot saved: {snapshot_path}")
        return snapshot_path
    def get_alert_count(self):
        return self.alert_count
```

MAIN.py

```
import argparse
import time
import cv2
import sys
from video_capture import VideoCapture
from analyzer import AggressionAnalyzer
```

```
from state_machine import StateMachine
from visualizer import Visualizer
from alert_logger import AlertLogger
import config
def parse_args():
    parser = argparse.ArgumentParser(
        description="AlertVision — Real-Time Aggression Detection"
    )
    parser.add_argument(
        "--video", type=str, default=None,
        help="Path to video file. If not set, uses webcam."
    )
    parser.add_argument(
        "--interval", type=float, default=config.SAMPLE_INTERVAL_SEC,
                                help=f"Seconds between analysis frames (default:
{config.SAMPLE_INTERVAL_SEC})"
    )
    return parser.parse_args()
def main():
    args = parse_args()
    # ── Initialize components ──────────────────────────────────────
    source = args.video if args.video else config.WEBCAM_INDEX
    capture = VideoCapture(source)
    analyzer = AggressionAnalyzer()
    fsm = StateMachine()
```

```
visualizer = Visualizer()
logger = AlertLogger()
print("=" * 55)
print("  AlertVision — Real-Time Aggression Detection")
print("=" * 55)
print(f"  Source   : {'Webcam' if args.video is None else args.video}")
print(f"  Interval : {args.interval}s between analyses")
print(f"  Model    : {config.GEMINI_MODEL}")
print("=" * 55)
print("  Controls: Q/ESC=Quit | SPACE=Pause | S=Screenshot")
print("=" * 55)
# ── Main loop ──────────────────────────────────────────────────────
last_analysis_result = None
last_analysis_time = 0
paused = False
frame_count = 0
with capture:
    while True:
        # Handle keyboard input
        key = cv2.waitKey(1) & 0xFF
        if key in (ord("q"), 27):  # Q or ESC
            print("\n[AlertVision] Shutting down...")
            break
        elif key == ord(" "):  # Space to pause
            paused = not paused
```

```
    print(f"[AlertVision] {'Paused' if paused else 'Resumed'}")
    continue
elif key == ord("s"):  # Screenshot
    if last_analysis_result:
        logger.log_alert(frame, last_analysis_result)
if paused:
    continue
# Read frame
success, frame, preprocessed = capture.read_frame()
if not success:
    if args.video:
        print("[AlertVision] Video ended.")
        break
    continue
frame_count += 1
now = time.time()
# ── Sample frame for analysis ──────────────────────────
if now - last_analysis_time >= args.interval:
    last_analysis_time = now
    print(f"\n[AlertVision] Analyzing frame #{frame_count}...")
    # Send preprocessed frame to Gemini
    last_analysis_result = analyzer.analyze_frame(preprocessed)
    # Update FSM
    state = fsm.update(last_analysis_result)
    # Check if alert should be triggered
```

```
            if fsm.should_alert():
                logger.log_alert(frame, last_analysis_result)
            status = last_analysis_result.get("status", "normal")
            conf = last_analysis_result.get("confidence", 0.0)
            desc = last_analysis_result.get("description", "")
            print(f"  → Status: {status} | Confidence: {conf:.0%}")
            print(f"  → {desc}")
        # ── Draw visualization ──────────────────────────────
        display_frame = frame.copy()
        if last_analysis_result:
            fsm_display = fsm.get_display_state()
            visualizer.draw(display_frame, last_analysis_result, fsm_display)
        else:
            visualizer.draw_waiting(display_frame)
        # Show frame
        cv2.imshow("AlertVision", display_frame)
    cv2.destroyAllWindows()
    print(f"\n[AlertVision] Total alerts: {logger.get_alert_count()}")
    print("[AlertVision] Done.")
if __name__ == "__main__":
    main()
```